

Module-I:

IoT Solution Models

Models applied in IoT solutions, Semantic models for data models, Application of semantic models, information models, information models to structure data, and relationships between data categories.

By

Dr Shola Usharani

Introduction to IOT solution Models and Architectures (Models applied in IoT solutions)

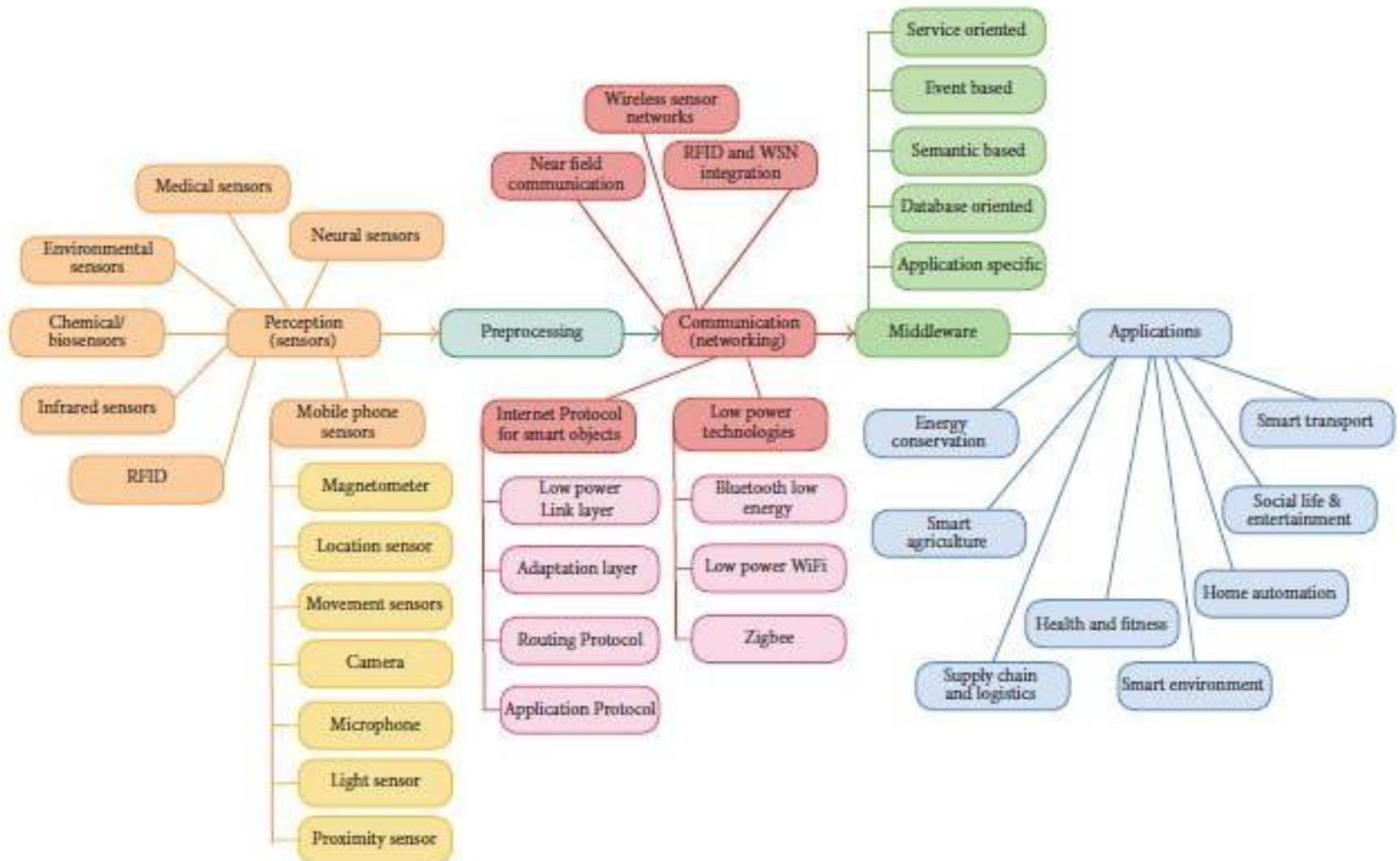
Taxonomy and reference model of IOT

- Taxonomy is defined as the abstract level for the collection of various domains.
 - Generally the abstract levels are defined as interfaces to the systems
- The reference model will cover the concepts, relationships, axioms among the entities of one particular context using these references.
- The required components include performance, security, functionality, deployment procedures.

Requirements of IoT layered taxonomy technology

- Bandwidth allocation should be flexible (from very low to high);
- The devices should perform extremely and cost for each node should be very low;
- The communication layers should be flexible and programmable (P2P, local, central);
- Most of the applications are with low latency;
- The reliability of the applications should be extremely high .
- Need to provide robust privacy and security.
- Meeting the non-functional elements like sustainability.

Taxonomy of IoT technologies



Reference Models

- It is a structured framework to understand, design, and implement IoT systems
- Reference models act as blueprints for developers, engineers, and architects, ensuring interoperability, scalability, and efficient resource utilization across IoT systems.
- Various IoT Reference Models
 - IoT-A Reference Model (Internet of Things Architecture) by EU-funded IoT-A project
 - ISO/IEC IoT Reference Architecture (ISO/IEC 30141)
 - ITU-T IoT Reference Model (International Telecommunication Union (ITU)).
 - Industrial Internet Consortium (IIC) Reference Architecture.
 - Open Systems Interconnection (OSI)-like IoT Reference Model

Various components of Reference Models

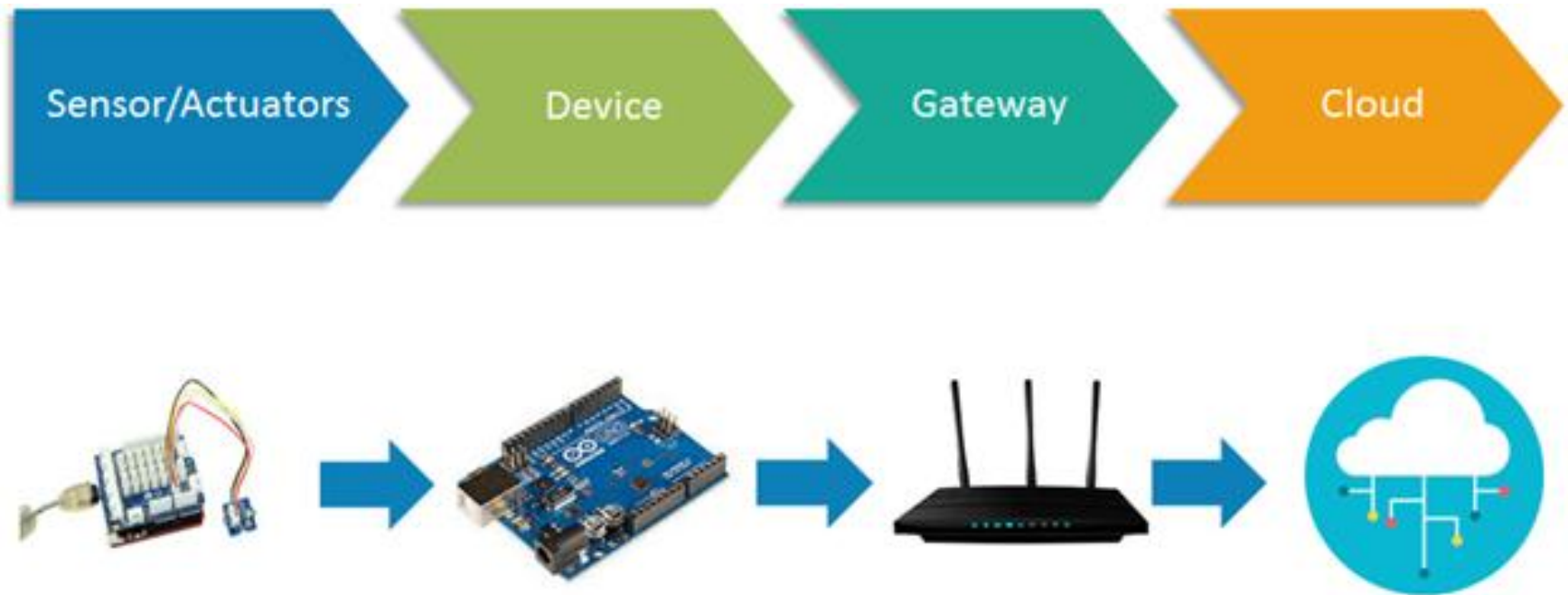
Sl.No	Reference Model	Components
1	IoT-A Reference Model	Business layer, Service Layer, Virtual Entity Layer, Communication Layer
2	ISO/IEC IoT Reference Architecture (ISO/IEC 30141)	Entities and Actors, Functional Viewpoint, Control Viewpoint, Physical Viewpoint
3	ITU-T IoT Reference Model	Perception Layer, Network Layer, Application Layer, Support Layer, Security Layer
4	Industrial Internet Consortium (IIC) Reference Architectur	Business Layer, Usage Viewpoint, Functional Viewpoint, Implementation Viewpoint
5	Open Systems Interconnection (OSI)-like IoT Reference Model	Perception Layer, Network Layer, Edge Layer, Application Layer

Requirements of IoT Architectures

- RFID established through various technologies
- Miniaturized objects as sensors to support the technology as smart.
- The solution of IoT like communication and tagging are well- developed for manufacturing and logistics.
- Business benefits for tracking asset and supply chain management.
- same solutions may not applicable to all other solutions.
 - Need to consider Interoperable, application area.
- It should satisfies the domains of all applications
- Common ground or an abstract level pattern that combines many applications need to consider
 - That framework is known as reference model

IoT Architecture

Components of IoT Architecture



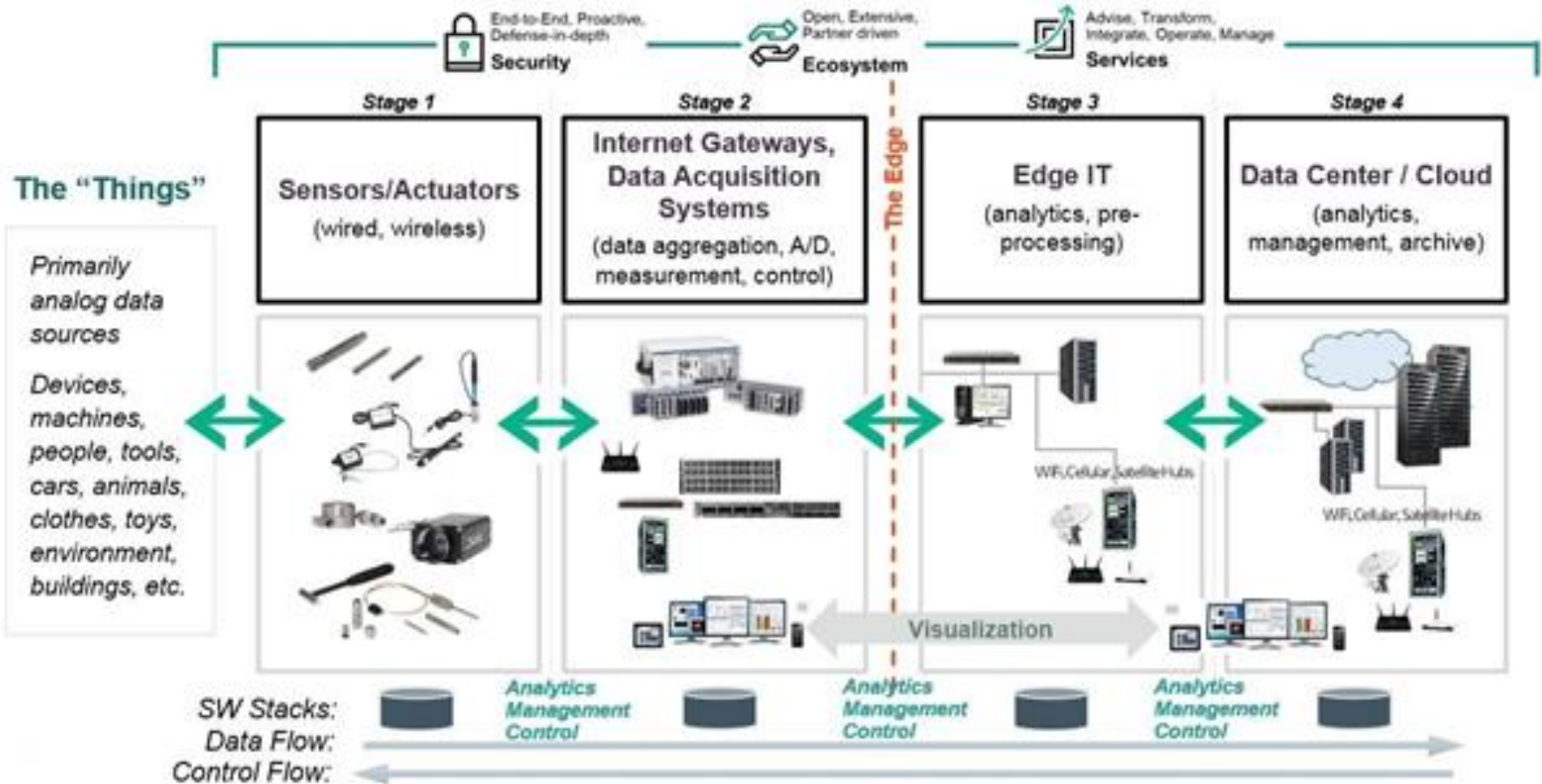
Source: <https://www.javatpoint.com/iot-architecture-models>

- Components of IoT Architecture
 - **Sensors/Devices**
 - **Gateways and Networks**
 - **Cloud/Management Service Layer**
 - **Application Layer**

Stages of IoT Solutions Architecture

- There are several layers of IoT built upon the capability and performance of IoT elements that provides the optimal solution to the business enterprises and end-users.
- The IoT architecture is a fundamental way to design the various elements of IoT, so that it can deliver services over the networks and serve the needs for the future.
- Following are the primary stages (layers) of IoT that provides the solution for IoT architecture.

The 4 Stage IoT Solutions Architecture



- **Sensors/Actuators:** Sensors or Actuators are the devices that are able to emit, accept and process data over the network.
- These sensors or actuators may be connected either through wired or wireless.
- This contains GPS, Electrochemical, Gyroscope, RFID, etc.
- Most of the sensors need connectivity through sensors gateways.
- The connection of sensors or actuators can be through a
 - Local Area Network (LAN) or
 - Personal Area Network (PAN). Or
 - Body Area Network (BAN) or
 - Wide Area network (WAN)

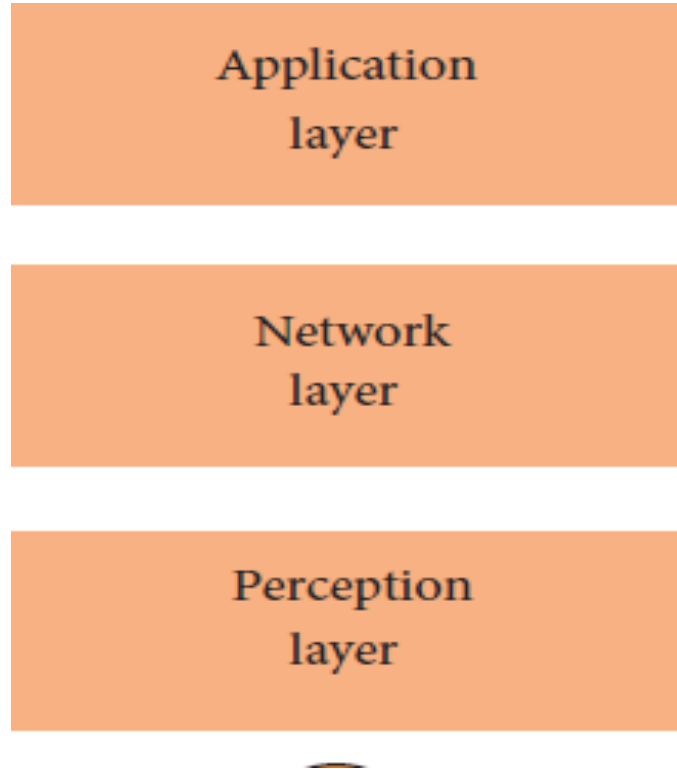
- **Gateways and Data Acquisition:** As the large numbers of data are produced by this sensors and actuators need the high-speed Gateways and Networks to transfer the data.
- This network can be of type Local Area Network (LAN such as WiFi, Ethernet, etc.), Wide Area Network (WAN such as GSM, 5G, etc.).

- **Edge IT:** Edge in the IoT Architecture is the hardware and software gateways that analyze and pre-process the data before transferring it to the cloud.
 - If the data read from the sensors and gateways are not changed from its previous reading value then it does not transfer over the cloud, this saves the data used.
- **Data center/ Cloud:** The Data Center or Cloud comes under the Management Services which process the information through analytics, management of device and security controls.
- Beside this security controls and device management the cloud transfer the data to the end users application such as Retail, Healthcare, Emergency, Environment, and Energy, etc.

Various IoT Architectures

- Three Layer Architecture
- Five layer Architecture
- Cloud and Fog Architecture
- Social IoT

Three Layer Architecture



Three Layer Architecture

- ***Perception layer*** : It senses the data from the physical parameters of the sensors. The sensed information of the environment is gathered from smart objects.
- ***Networks layer*** : It is used for transmitting and processing the gathered data. It is used for connecting smart objects, network devices, and servers.
- ***Application layer*** : It is used for providing the application services that are specific to the user. It specifies various applications that are deployed in the IoT environment

Disadvantages

- The three-layered architecture just defines the basic idea about the IoT, but it does not cover other aspects of IoT

Five layer Architecture

Business layer

Application layer

Processing layer

Transport layer

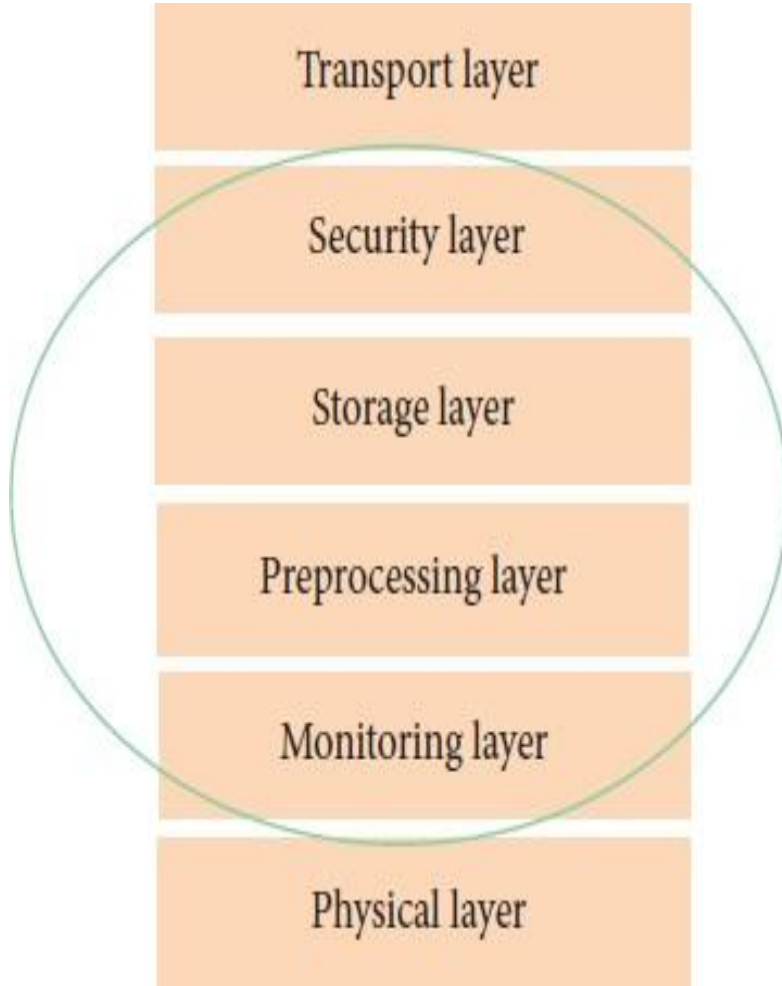
Perception layer



Five layer Architecture

- **Transport layer:** The transferring networks include RFID, 3G, LAN, Wireless, and NFC.
- **Processing layer:** it is the middle layer among the 5-layer mode and is also treated as a middleware layer.
 - It stores, analyses and processes the large amounts of information obtaining from the transport layer. It also provides services to the lower-level layer.
 - the processing layer gathers data from the perception or transport layer and similarly from transport layer to the processing layer.
 - It includes technologies like **databases, big data processing, and cloud computing.**
- the application layer is used the same as represented in three-layered architecture.
- **Business layer:**
 - it manages the entire IoT system::utilizes the data generated by the devices and sensors within the IoT ecosystem.
 - It includes applications, business models, security and privacy.
 - Key responsible: Data Integration and Management, Analytics and Insights, User Interfaces, Security and Compliance

Cloud and Fog Architecture



- Applications that need data processing through a **large centralized cloud computer** where they need scalability and flexibility.
- offers services like **platform infrastructure, application software, storage tools, and data mining tools, machine learning, and visualization tools.**
- fog based architectures that include monitoring, pre-processing, storage, and the physical, transport layers in between with security layer.

Cloud and Fog Architecture

- Monitoring Layer :Resources, power, services and responses are various components processed and displayed.
- The pre-processing layer: executes various operations on sensor data like cleaning, processing, and analytics.
- The temporary storage layer delivers storage jobs such as data duplication, distribution, and storage.
- Finally, data integrity, privacy and security components like encryption/decryption are performed at security layer

Social IoT (SIOT)

- refers to the integration of social interactions and behaviors into the architecture of Internet of Things (IoT) systems.
- It involves incorporating social elements such as user preferences, social relationships, and collaborative data sharing.
- Key Components of Social IoT Architecture
 - Social Layer
 - Data Layer
 - Interaction Layer
 - Application Layer

Social Layer

- User Profiles and Preferences
 - It is about user preferences, interests and behavioural data. Represented in terms of social graphs.
- Social Interaction Models
 - It is about user interaction with devices with each other. It is about sharing data, coordination actions
- Reputation Systems
 - It is to bring good reputation for a task or business.
 - It evaluates and manage the trust among users and devices

Data Layer

- **Social Data Integration:** Incorporates data from social networks, online communities, and other platforms to enhance the IoT experience.
- **Context-Aware Data Processing:** Data is processed by considering social context like user behavior and preferences.

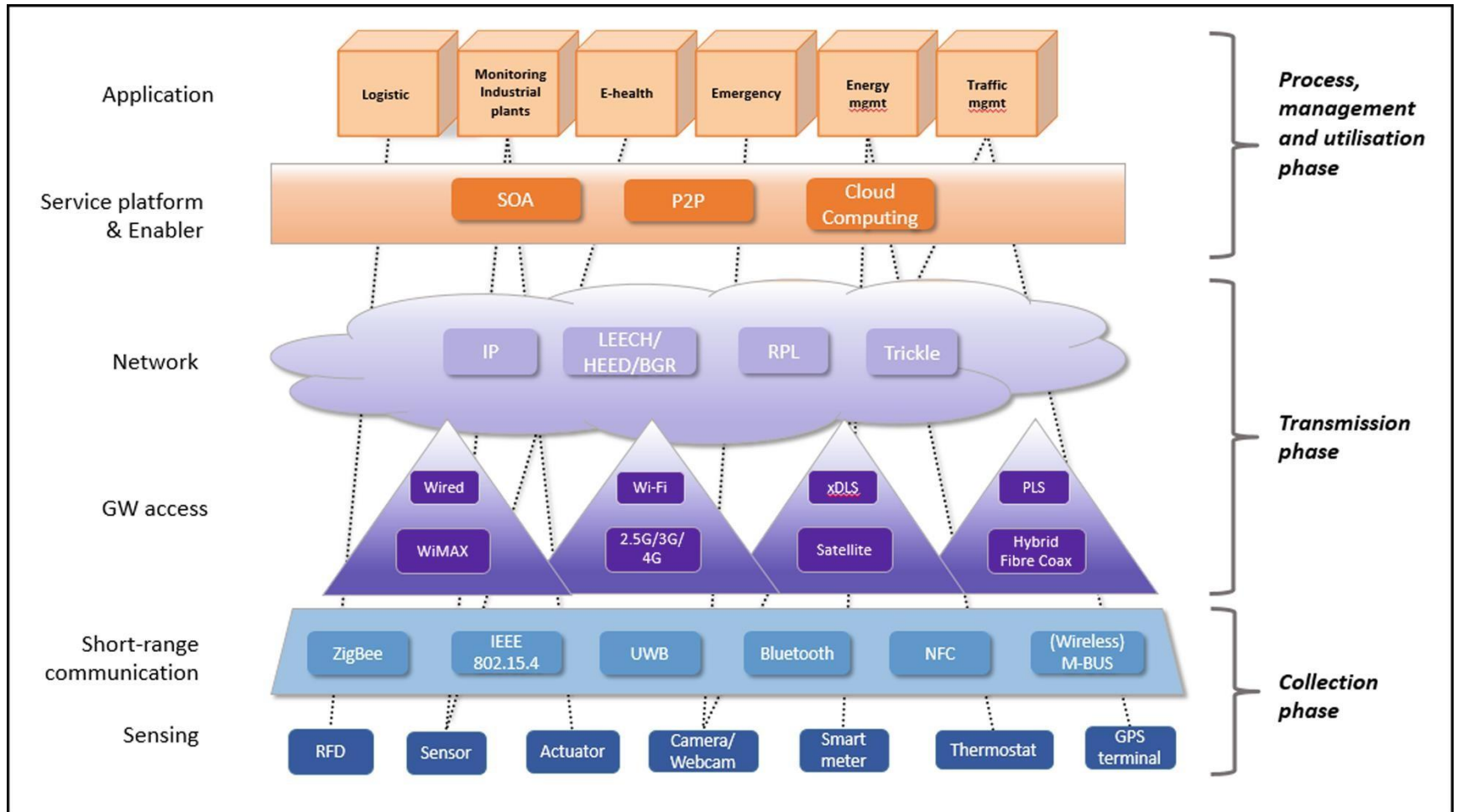
Interaction Layer

- Collaboration Mechanisms: sharing data, sending notifications, and coordinating actions across devices based on social agreements.
- Social Feedback: provide feedback on device performance, services, or recommendations. This feedback can be used to improve future interactions and services.

Application Layer

- Social IoT Applications: Applications that leverage social interactions to provide new services like, social-based energy management, shared home devices management, and collaborative health monitoring.
- Social Community Management: maintaining privacy and security settings for shared IoT devices.
- **Gamification:** Using game elements to encourage positive behaviors, such as energy saving or physical activity

Generalised IoT Solution Model



Issues & challenges of IoT

Issues

- Security
- Privacy
- **Interoperability and Standards (data modelling).**
- Legal, Regulatory and Rights
- Emerging economics and development

Understanding Real world problems

- Smart Cities asks for water leak, street faults, broken street lights, and potholes.
- Need a good communication and interaction channel between citizens and the administration in improving the quality of urban life of citizen
- Solution
 - best practices of linking open data to describe all issues, and then integrate them in the dataset

Need of Data Modelling for IoT Solutions

- IoT opportunities in **various categories** ranging from consumer home need **applications** to industry based applications
 - Home automation, factory, personal monitoring, health and fitness, smart city.
- Causing connectivity of **billions of newly connected devices**, need to **handle the data connected** virtually and physically
- Need to find ways to connect, manage, analyse and to **share the data with** different organizations.
- Distribution of data management due to heterogeneous of systems.
 - Heterogeneity introduces challenges of data interoperability for interchanging the data between devices , sensors of various vendors and between supply chain partners etc.,

Advantages of Data modelling

- Offers an approach which could more efficiently **describe, interpret, analyze** and share data among heterogeneous IoT applications and devices
- It helps to **learn, optimize processes** and understand the hidden rules of our world
- It helps in **obtain knowledge/insights** beyond the original applications

Examples of data API models

- Smart Appliances **RE**ference(SAREF) provides a shared reference framework model for home appliances.
 - to ensure interoperability between smart appliances within the Internet of Things (IoT) ecosystem
 - Developed by ETSI (European Telecommunications Standards Institute) in collaboration with industry stakeholders
 - SAREF provides a common language and data model for smart appliances to communicate effectively across different manufacturers and platforms.
- **Open Geospatial Consortium** for geosciences and environment domains.
- The **Open Connectivity Foundation** specifies data models based on vertical industries such as automotive, healthcare, industrial and the smart home.
- The **World Wide Web Consortium Thing** description provides some vocabularies to describe physical things.
 - It will create standards and frameworks that enable devices to interact seamlessly over the web. A significant initiative by W3C for IoT is the **Web of Things (WoT)** framework.
- **Schema.org** operates as a collaborative community and it aims to provide more general and broad vocabularies and schemas for structured data on the internet

Data Model provides standardization

- It is to create a body of maximally reusable standards to enable IoT applications across different verticals
- Future research works in IoT data modelling
 - To manage data models across application verticals and domains sets the stage for the **next phase of the IoT industry's growth**

Models applied in IoT Solutions

- Semantic data modelling
- RDF graph representation
- Ontology structure
- Information modelling

Semantic Web is the way of
understanding the IoT data that
connecting the Real World

SEMANTIC DATA MODEL

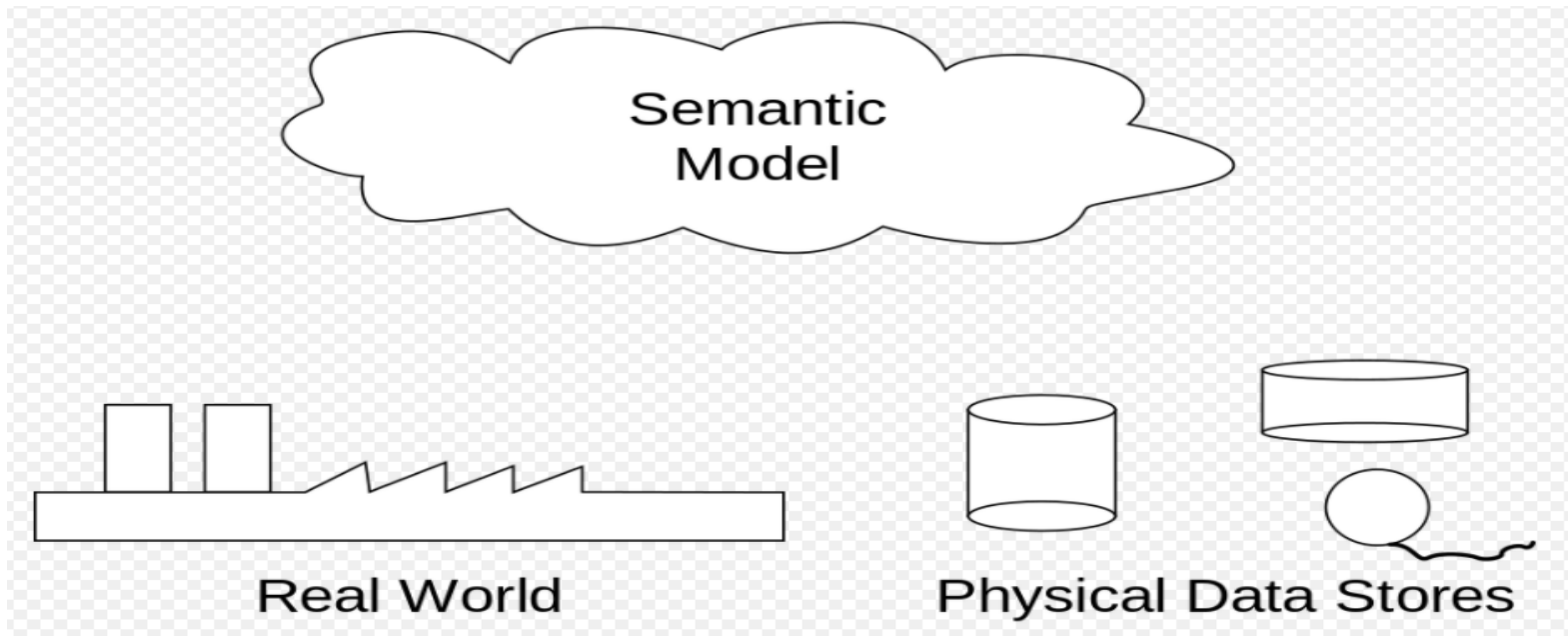
- semantic data model is an abstraction that defines **how the stored symbols (the instance data) relate to the real world**.
- It is the capability to express and exchange information which enables parties to interpret meaning (semantics) from the instances, without the need to know the meta-model.
- It provides the relations between data elements, whereas higher order relations are expressed as collections of binary relations.
- It uses concepts like entities, attributes, and relationships to represent real-world objects in a meaningful way
- include the kinds of relationships between the various data elements, such as <is located in>
 - For example: the Eiffel Tower <is located in> Paris.

Semantic models for IoT

- Data models and semantics are a key aspect for the data in **cross-domain applications and to obtain knowledge/insights** beyond the original applications.
- Semantic models often use ontologies or semantic standards to enhance interoperability and understanding across systems
- Provides integrating the data into the data set.
 - Examples : Dbpedia, Geonames and FreeBase

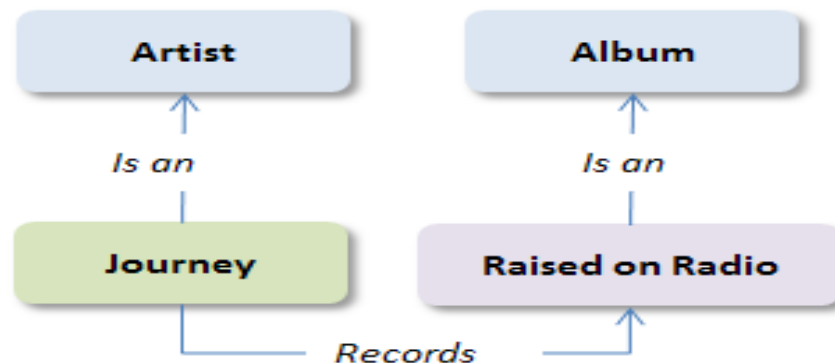
Contd.,

- **semantic data model** as a conceptual diagram the data as it relates to the real world.



Semantic Data Model Example

- Generating next-generation music app Semantic model
 - Example :
 - Journey is an artist; an artist records an album; *Raised on Radio* is an album



Core concepts in Semantic Data Modelling

- Entities: Objects or concepts in the real world (e.g., a "Car," "Person," or "Building").
- Attributes: Properties or characteristics of an entity (e.g., a "Car" has attributes like "Color" and "Make").
- Relationships: Associations between entities (e.g., a "Person" drives a "Car").
- Ontology: A formal representation of concepts, their properties, and relationships in a domain.

DBpedia

- It Converts human-readable Wikipedia data into RDF for automated processing.
- **DBpedia** is to extract structured content from the information created in various Wikimedia projects
 - Data model for Wikimedia projects.
- It is available as an **open knowledge graph (OKG)** which is available for everyone on the Web.
 - A knowledge graph is a special kind of database which stores knowledge in a machine-readable form and provides a means for information to be collected, organized, shared, searched and utilized.
- This structured information is then made available on the World Wide Web by using a package of APIs 6

Various types of data models

TABLE 1. Different types of data models in a Smart City.

Category	Description	Publisher (Data owner)	Data access
Mobility	City Maps (Roads, street names, Points of Interests (POIs), stations, etc.)	Government, Private Companies and Organizations	OpenStreetMap (Open Data), APIs accessible (Google Maps) and proprietary/closed GIS such as ESRI
	Public transport schedules	Government and Private Companies	Open Data or Web-accessible content
	Transport authority updates and events (road status, road works...)	Government and Private Companies	Waze (APIs accessible), Web-accessible data and Open Data
	Car Parks/Parking meters	Government and Private Companies	APIs accessible and Open Data
	Traffic (Number of vehicles passing between two points and speed)	Government and Private Companies	Private data, APIs accessible and Open Data
	Opportunistic and Crowd monitoring	Government, Private Companies and Organizations	Nomadic Sensing (NOSE)
Environmental data	Air Quality (Air Quality Index - AQI, Particles concentration and gases concentration)	Government	Private data, APIs accessible and Open Data
	Temperature/Humidity	Government	Private data, APIs accessible and Open Data
	Noise level/Noise maps	Government	Private data, APIs accessible and Open Data
Tourism	Points of Interest, Time schedule, events, museums, bar/discos etc.	Cultural Groups	APIs and Web available data
Demographic	Population, Crowd Monitoring, Ages, Cultural Level, Economical Level	Government and organizations	Open Data
Co-creation (Citizens engagement)	Opinions, decisions, surveys, etc.	Government and organizations	Siddi (Organicity)

Readable data structures

- Tells how to convert the real word data into structured format.
- Principles to follow to share the machine readable IoT data on Web.
- OGD (Open Government Data)
 - is a philosophy- and increasingly a set of policies - that promotes transparency, accountability and value creation by making government data available to all
- LOD(Linked open Data)
 - is a set of design principles for sharing machine-readable interlinked **data** on the Web
 - a best practice of promoting the sharing and publication of structured data on the semantic Web
- RDF (**Resource Description Framework**)
 - RDF→publishing the data set (ontology structure as DBPedia, Geonames and Freebase)

Resource Description Framework(RDF)

- RDF is a framework for describing resources on the web
- RDF is designed to be read and understood by computers
- RDF is not designed for being displayed to people
- RDF is written in XML
- RDF is a part of the W3C's Semantic Web Activity
 - Describing properties for shopping items, such as price and availability
 - Describing time schedules for web events
 - Describing information about web pages (content, author, created and modified date)
 - Describing content and rating for web pictures

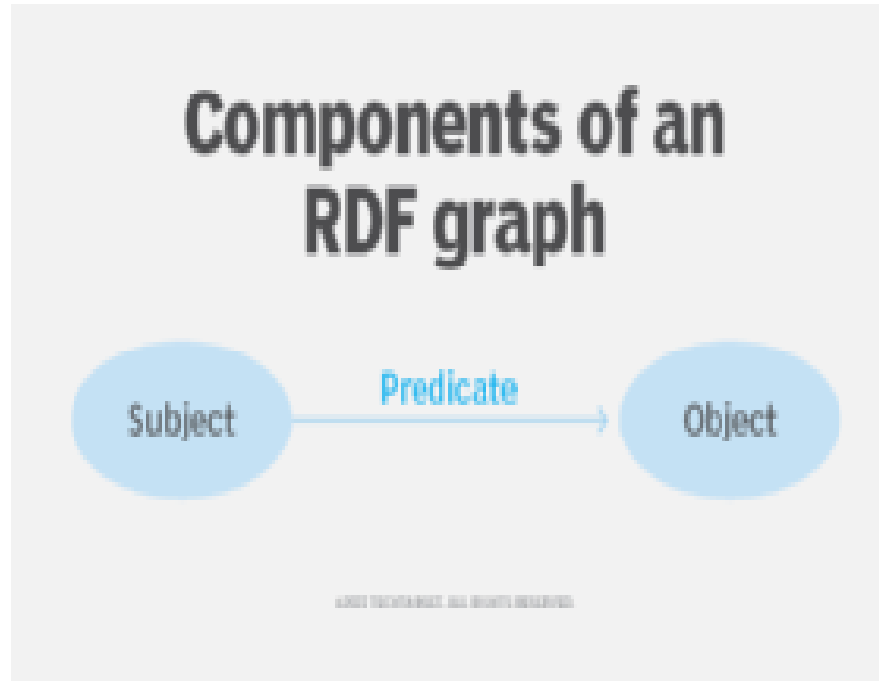
RDF

- The semantic web is based on the use of the RDF framework to organize information based on meanings.
- RDF statements express relationships between resources, such as the following:
 - documents
 - physical objects
 - people
 - abstract concepts
 - data objects
- A collection of RDF statements about related entities can be used to construct an RDF graph that shows how those entities are related.

Contd.,

- RDF is a standard way to make statements about resources. An RDF statement consists of three components, referred to as a *triple*:
 1. **Subject** is a resource being described by the triple.
 2. **Predicate** describes the relationship between the subject and the object.
 3. **Object** is a resource that is related to the subject.
- The subject and object are nodes that represent things. The predicate is an arc, because it represents the relationship between the nodes

Components of RDF graph



Various forms of RDF statements

- The World Wide Web Consortium ([W3C](#)) maintains the standards for RDF, including the foundational concepts, semantics and specifications for different formats.
- The first syntax defined for RDF was based on the Extensible Markup Language ([XML](#)).
- Other syntaxes are now more commonly used, including Terse RDF Triple Language (Turtle), JavaScript Object Notation for Linked Data ([JSON](#)-LD) and N-Triples.

How RDFS used

- RDF statements can be incorporated into Hypertext Markup Language webpages or stored in separate files and linked to data in web content.
- When RDF was first specified, RDF statements were incorporated into XML documents linked with web content

- The subject of the statement above is:
<https://www.w3schools.com/rdf>
- The predicate is: author
- The object is: Jan Egil Refsnes

RDF representation

- Resource (Subject): The entity being described.
 - Represented as a URI (Uniform Resource Identifier) to uniquely identify it on the web.
 - Example: `<http://example.org/resource/AlbertEinstein>` (a unique reference for Albert Einstein).
- Property (Predicate):
 - Defines a characteristic or relationship of the resource. Represented as a URI to standardize the meaning of the property.
 - Example: `<http://example.org/property/birthPlace>` (a property defining the birthplace of a person).
- Property Value (Object):
 - The value of the property, which could be: A literal (e.g., text, number, date): "Ulm".
 - Another resource (URI): `<http://example.org/resource/Ulm>`.

Example 1: RDF Triple

- **Statement:**
- "Albert Einstein was born in Ulm.,,"
- RDF Representation (in Turtle syntax):
 - `<http://example.org/resource/AlbertEinstein>`
`<http://example.org/property/birthPlace>`
`<http://example.org/resource/Ulm> .`

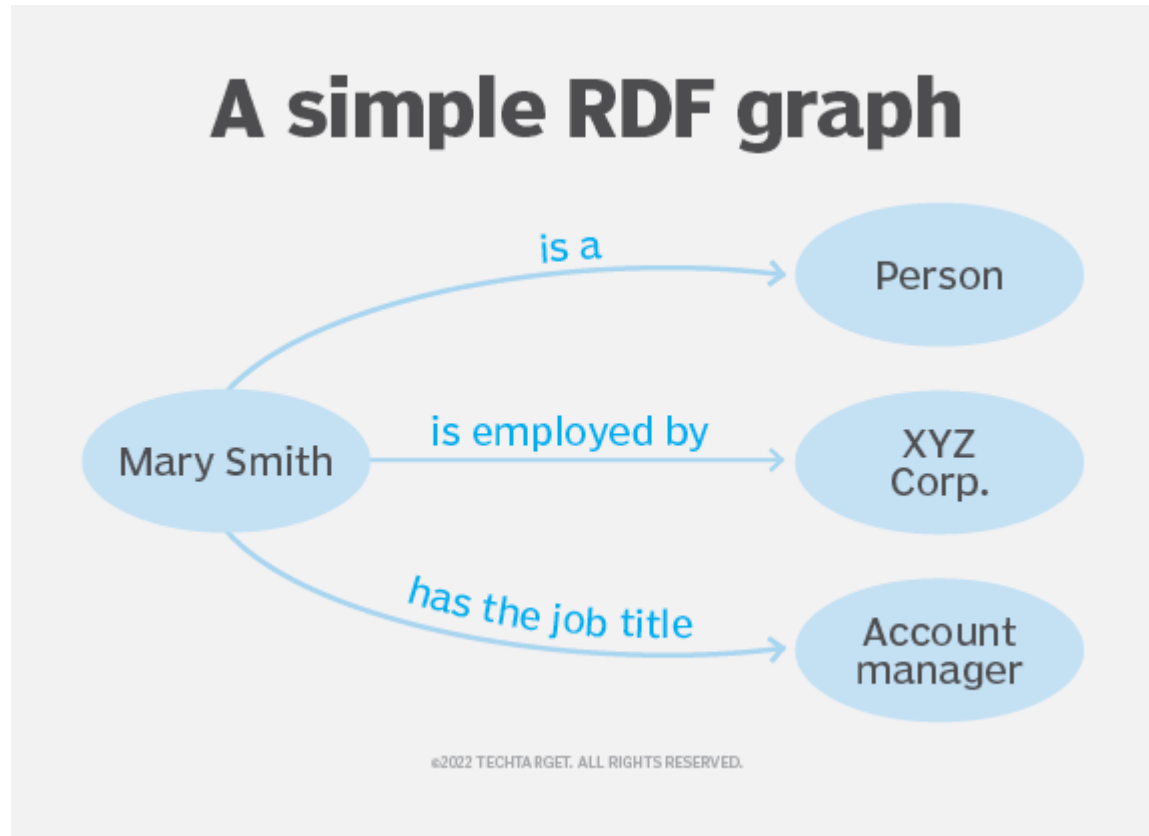
Example 2: Resource with a Literal Property Value

- **Statement:**
- "Albert Einstein was born on March 14, 1879."
- RDF Representation:
- `<http://example.org/resource/AlbertEinstein>`
`<http://example.org/property/birthDate>`
`"1879-03-14"^^xsd:date`

Example 3: Resource with Multiple Properties

- **Statement:**
- "Albert Einstein was born in Ulm, was a physicist, and was born on March 14, 1879."
- RDF Representation:turtle
 - <http://example.org/resource/AlbertEinstein>
 - <http://example.org/property/birthPlace>
 <http://example.org/resource/Ulm> ;
 - <http://example.org/property/occupation>
 "Physicist" ;
 - <http://example.org/property/birthDate> "1879-03-14"^^xsd:date .

RDF for multiple triples (Try this)



More on RDF

- <https://www.techtarget.com/searchapparchitecture/definition/Resource-Description-Framework-RDF>

Drawbacks of RDF

- Standardization vocabulary using RDF resources is difficult
- Choosing most appropriate for query language processing for RDF depends on the needs of an application.

Ontology

- representation of terms and their interrelationships is called an ontology.
- more facilities for expressing meaning and semantics

ontology structure

- It represents a formal, hierarchical model of knowledge within a specific domain.
- It defines **concepts**, **relationships**, and **properties** to organize and standardize information

OWL

- OWL Web Ontology Language.
- OWL is intended to be used when the information contained in documents needs to be processed by applications, as opposed to situations where the content only needs to be presented to humans.
- OWL can be used to explicitly represent the meaning of terms in vocabularies and the relationships between those terms
- OWL has more facilities for expressing meaning and semantics than XML, RDF, and RDF-S

Various ontology structures

- M3-Lite
- W3C SSN
- oneM2M Ontology SAREF Ontology
- SAREF Ontology
- FIWARE and ETSI ISG CIM
- OMA LwM2M and IPSO smart Objects

Components of Ontology structure

- Classes: Represents a group or category of things (also called *entities* or *instances*),
 - Examples : Person, Teacher , student
- Relationships (Object Properties) : Describes how different classes relate to each other.
 - Hierarchical Relationships (Is-a): Teacher is a Person
 - Part-Whole Relationships (Has-a): Car has an Engine
 - Associative Relationships: Teacher teaches Subject.
- Attributes (Data Properties): Represents specific characteristics of a class or instance.
 - Class: Person
 - Attribute: hasName → Data type: String
 - Attribute: hasAge → Data type: Integer
- Instances : Concrete examples of classes
 - Class: Person
 - Instance: John Doe
 - Attribute values: hasName = "John Doe", hasAge = 30

Contd.,

- Hierarchy: Defines how classes and subclasses are organized.
 - LivingBeing
 - Animal
 - Mammal
 - Dog
- Rules and Constraints : Logical expressions that define how concepts relate or how data is validated.
 - Rule: Every Student must enroll in at least one Course.
 - Constraint: has Age must be greater than 0.
- Ontology Languages
 - RDF (Resource Description Framework)
 - OWL (Web Ontology Language)
 - RDFS (RDF Schema)

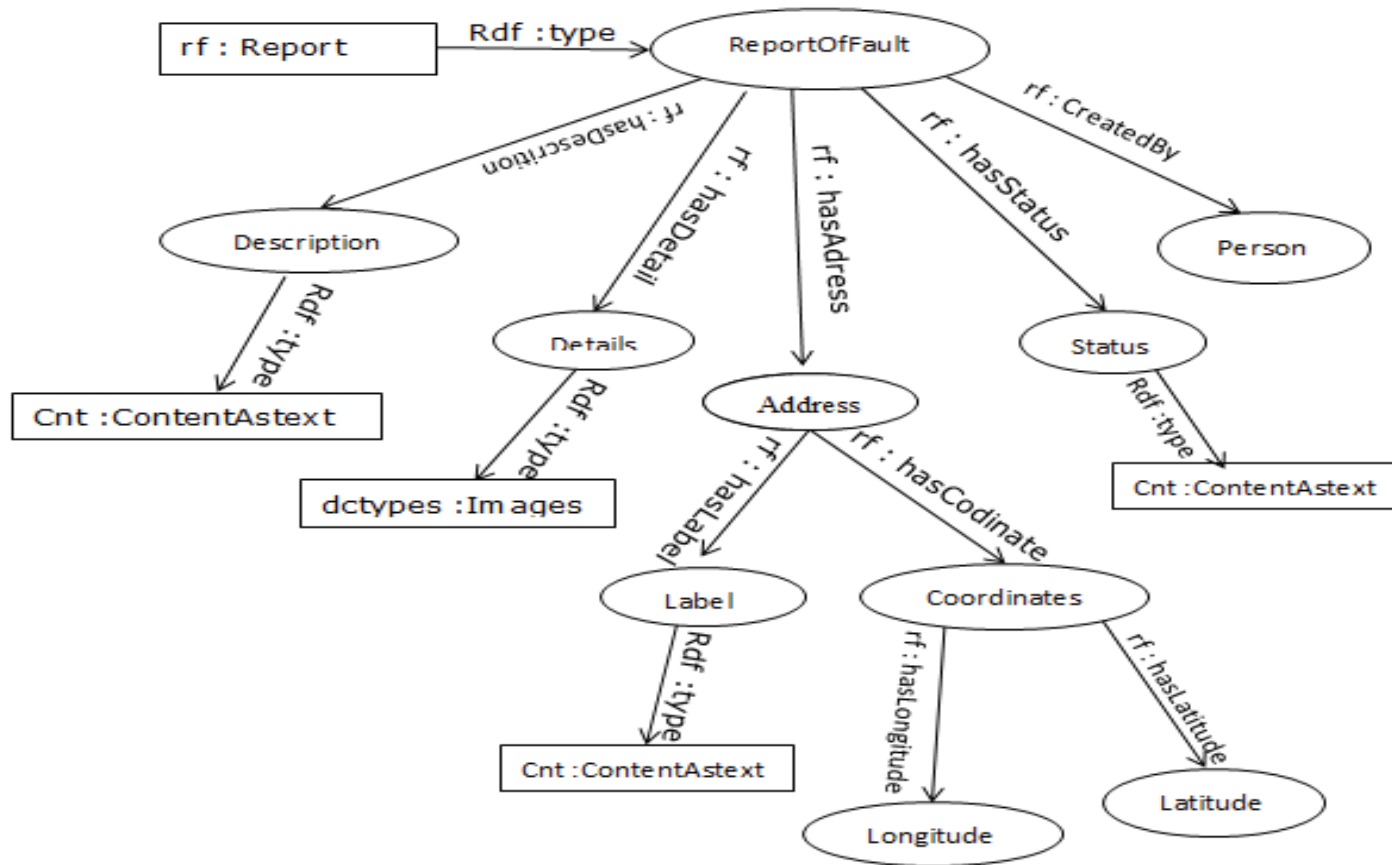
Example of smart city fault report generation

Adress Souk Ahras GO	
Latitude	Longitude
36.2807608	7.9381485
	

E-mail		Phone number	
Date		Hour	
First name		First name	
Adress			
Type of fault	☐ Water leak ☐ Street faults ☒ Broken street lights ☐ Potholes		
<u>Comment</u>			
<u>Photos</u>			
Photo 1	Load...		
Photo 2	Load...		
Photo 3	Load...		
Photo 4	Load...		
Send			

Ontology structure

- [Ontology.docx](#)



Application of semantic models

- Smart city data integration
 - the standardization and data modelling actions that are working towards defining a common semantic descriptions for smart city data to enable cross-domain and advanced services development in smart cities
 - Detecting the fault raised by the citizens
 - semantic data model for managing and resolving the problems that exist in cities as water leak, street faults, broken street lights, and potholes

Steps

- Obtaining the data in XML
- Transforming the information into RDF from XML
- Constructing the ontology structure or RDF graph from the RDF data
- Saving the data into database
- Query the data for further processing

HTML Input

Adress

Latitude Longitude



Google

Données cartographiques Conditions d'utilisation Signaler une erreur cartographique

E-mail Phone number

Date Hour

First name First name

Adress

Type of fault ☐ Water leak ☐ Street faults ☒ Broken street lights ☐ Potholes

Comment

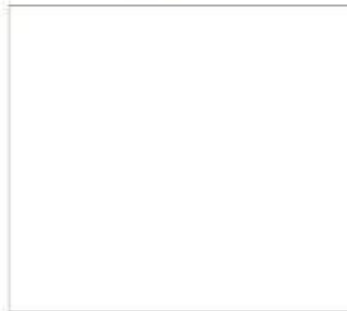
Photos

Photo 1

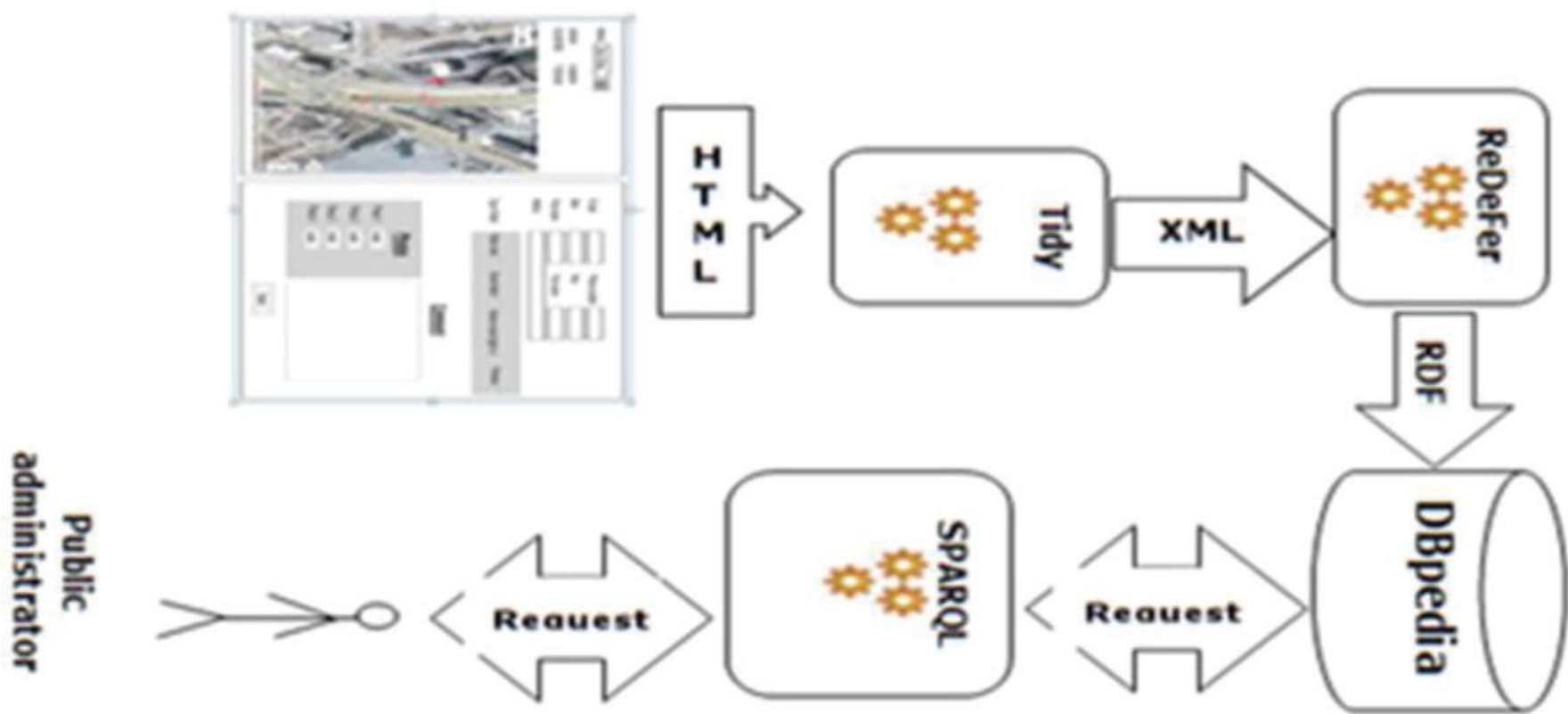
Photo 2

Photo 3

Photo 4



Data of RDF to DBPedia



Try to create Ontology structure for the given code

- Ontology: university
- Class: Person
- Class: Student
 - SubClassOf: Person
- Class: Teacher
 - SubClassOf: Person
- Class: Course
- ObjectProperty: teaches
- Domain: Teacher
- Range: Course
- ObjectProperty: enrolledIn
- Domain: Student Range: Course
- DataProperty: hasName
- Domain: Person
- Range: xsd:string
- DataProperty: hasAge
- Domain: Person
- Range: xsd:integer
- DataProperty: courseCode
- Domain: Course Range: xsd:string

- Individual: John_Doe
 - Types: Student
 - Facts:
 - hasName "John Doe"^^xsd:string
 - hasAge 21^^xsd:integer
 - enrolledIn CS101
- Individual: Jane_Smith
- Types: Teacher
- Facts:
 - hasName "Jane Smith"^^xsd:string
 - hasAge 40^^xsd:integer
 - teaches CS101
- Individual: CS101
- Types: Course
- Facts:
 - courseCode "CS101"^^xsd:string

Information modelling

Introduction

- Information Model provides the framework for **organizing your content** so that it can be **delivered and reused in a variety of innovative ways**.
- It allows **labelling** of your repository to enhance the search and retrieval of the information.
- Easily accessible the information.
- The Information Model is the **content-management** tool.

Why we need information model

- solves the problems described when it is **designed in the context of a content-management** system
- The model labels information according to the ways it will be accessed
- the framework needed **to make information accessible to experienced and inexperienced seekers**
- It reduces frustration and enhances productivity.
 - It means that people spend less time searching and more time using information resources
- Repeatability is reduced
 - It helps to ensure that resources are not rewritten or recreated

Example without modelled web page

- You can't tell how to get from the home page to the information you're looking for.
- on click on a promising link and are unpleasantly surprised at what turns up.
- You keep drilling down into the information layer after layer until you realize you're getting farther away from your goal rather than closer.
- Every time you try to start over from the home page, you end up in the same wrong place.
- You scroll through a long alphabetic list of all the articles ever written on a particular subject with only the title to guide you.

Example with information modelled web page

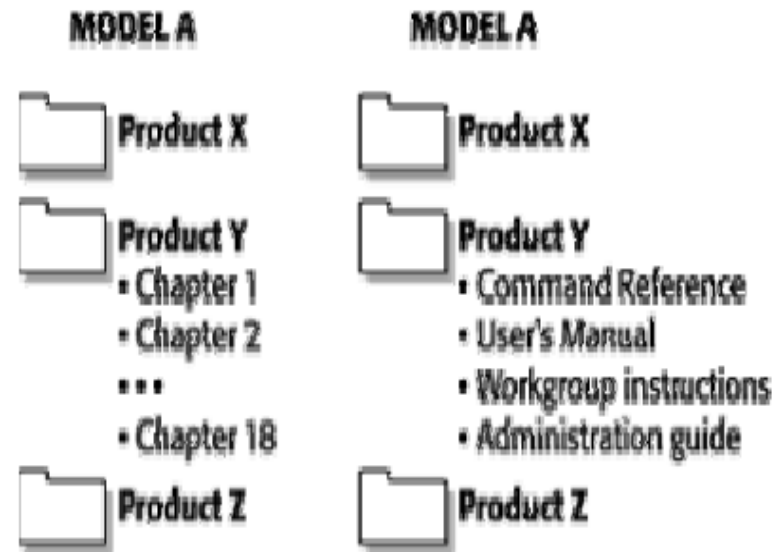
- On the home page, you notice promising links right away.
- Two or three clicks get you to exactly what you wanted.
- The information seems designed just for you because someone has anticipated your needs.
- You can read a little or ask for more – the cross-references are in the right places.
- Right away you feel that you're on familiar ground – similar types of information start looking the same.

Construction of Information models

- Static Information models
- Dynamic Information models

Static Information models

- To construct Information Mode in one particular logical use format.
 - hierarchical arrangement used in a file-management system
 - Functional departments within companies
 - categories such as employee benefits, employee demographic information, and so on.
 - The electronic filing system



Dynamic Information Models

- Changes in response to the needs of the users
 - Example : an online library “patron” would rearrange itself in response to a particular set of needs.
 - Let's say that a patron wants to “**find not only all the books written by Steven King** but also all the books and articles written about Steven King”.
 - In addition, the patron would like to know “more about mystery and horror writers in the second half of the twentieth century living in North America or in the United Kingdom”.

Information modelling languages

- ER
 - [ER Diagram MMORPG\(Information modeling\).svg](#)
- Semantic modelling techniques
- Unified Modelling language

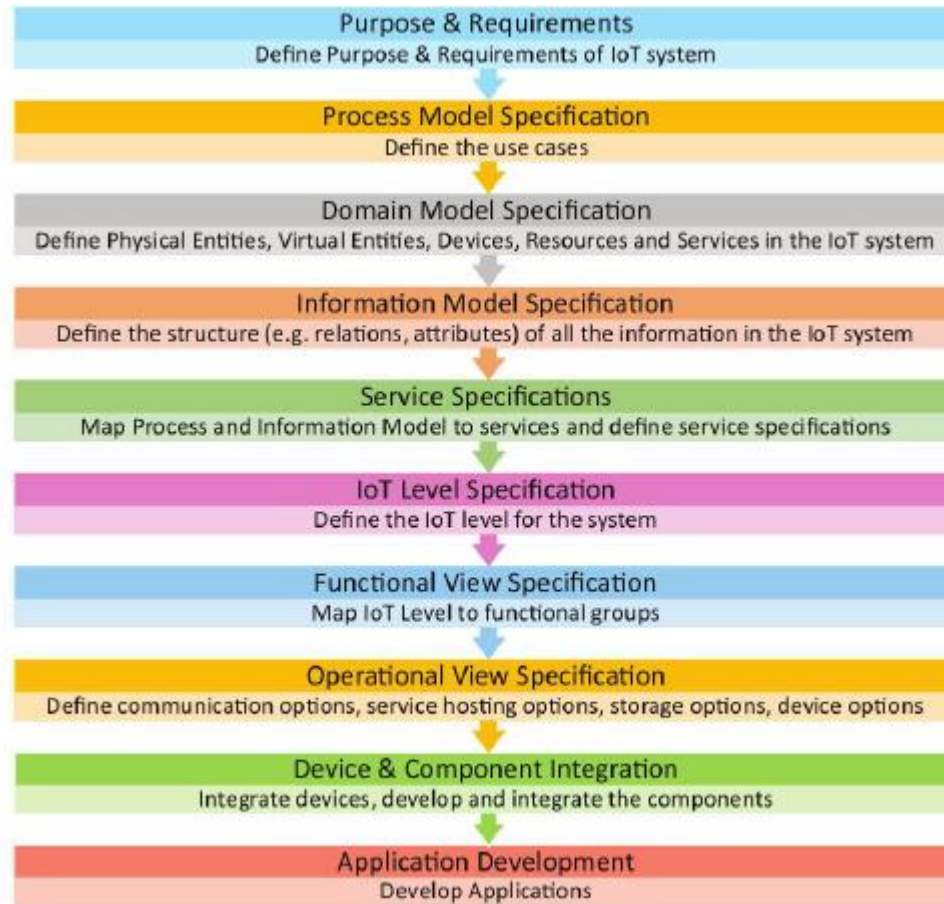
Building information modelling (BIM): Example

- It is a process supported by various tools, technologies and contracts involving the generation and management of **digital representations of data of physical and functional characteristics of places** and buildings.
- Building information models (BIMs) are **computer files not in proprietary formats** and containing proprietary data which can be extracted, exchanged or networked to support **decision-making regarding a built asset**.
- BIM software is used by individuals, businesses and government agencies **who plan, design ,construct, operate and maintain buildings and diverse physical infrastructures, such as water, refuse, electricity, gas, communication utilities, roads, railways, bridges, ports and tunnels**.

Advantages of BIM

- software tools developed for modelling building or **Building Description System**
- **Interoperability and BIM standards**
 - BIM software developers have created proprietary data structures in their software, data and files created by one vendor's applications may not work in other vendor solutions. To achieve interoperability between applications, neutral, non-proprietary or open standards for sharing BIM data among different software applications have been developed.
 - BIM is often associated with Industry foundation classes (IFCs) and secXML – data structures for representing information – developed by buildingSMAR.
 - IFC is recognized by the ISO and has been an official international standard, ISO 16739.

IoT System Design Methodology for data (Information modelling process)



Home automation system

- A home automation system, allowing and controlling of lights through an web based application. It two types of modes Auto mode and Manual mode to control the operation of lights

Purpose & Requirements Specification

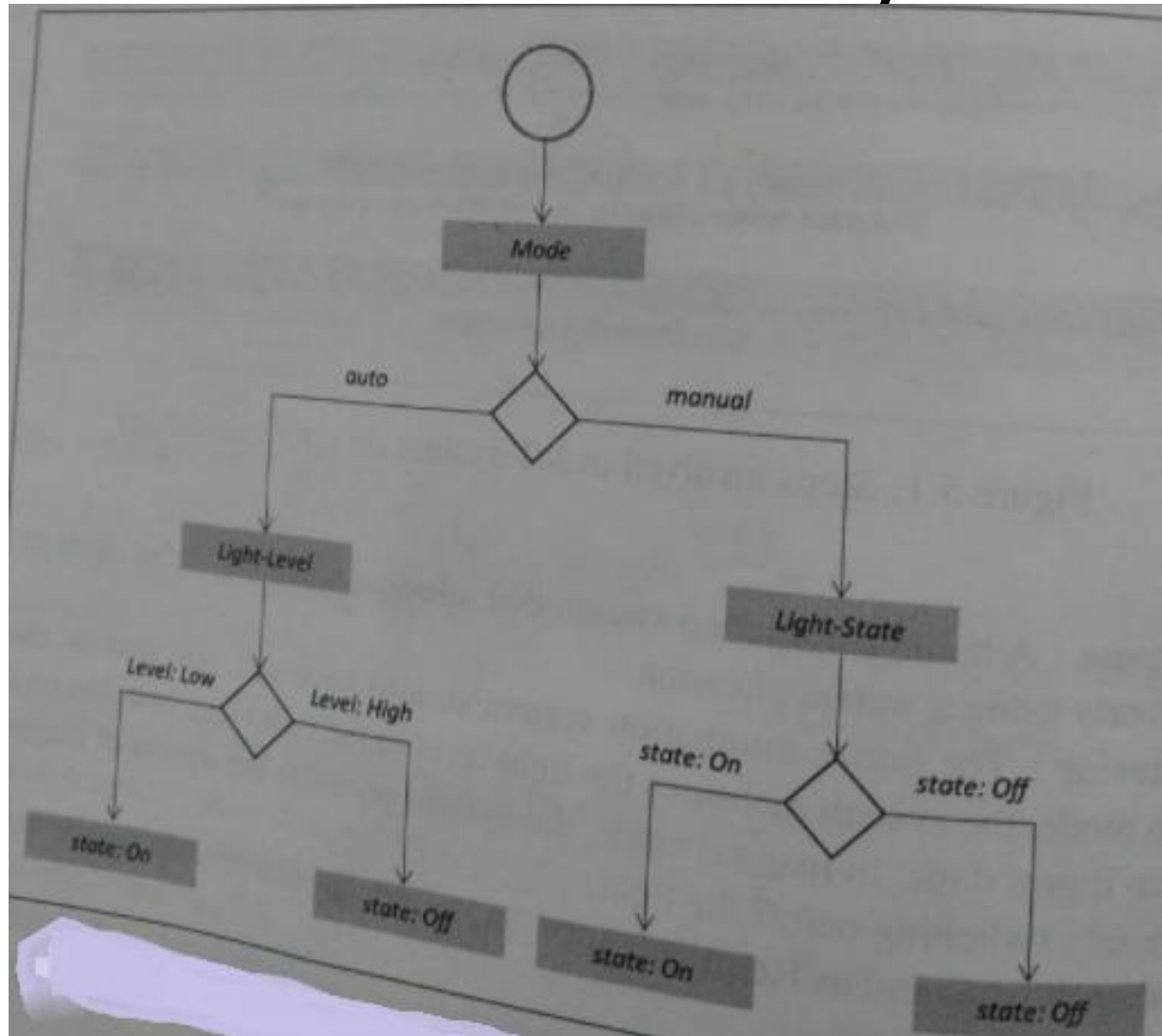
Purpose, behaviour and requirements like **data collection, data analysis, system management requirements**

- Purpose: for home automation system, allowing and controlling of lights through an web based application.
- Behaviour: two types of modes
 - Auto mode : measures the light level in the room and switches ON/OFF the light
 - Manual mode : provides option of manual or through remotely.
- System Management Requirements
 - Provision of control operations and remote monitoring.
- Data Analysis Requirement
 - Analysis of data locally.
- Application deployment requirement
 - Application is deployed locally but accessed remotely.
- Security Requirement
 - Use authentication capability.

Process Specification

- Defining the use cases based on step1
- Process representation using flow diagrams.
- Various symbols:
 - Circle : about start of the process
 - Decision box
 - Rectangle : state or attributes.

Process specification for home automation IoT system



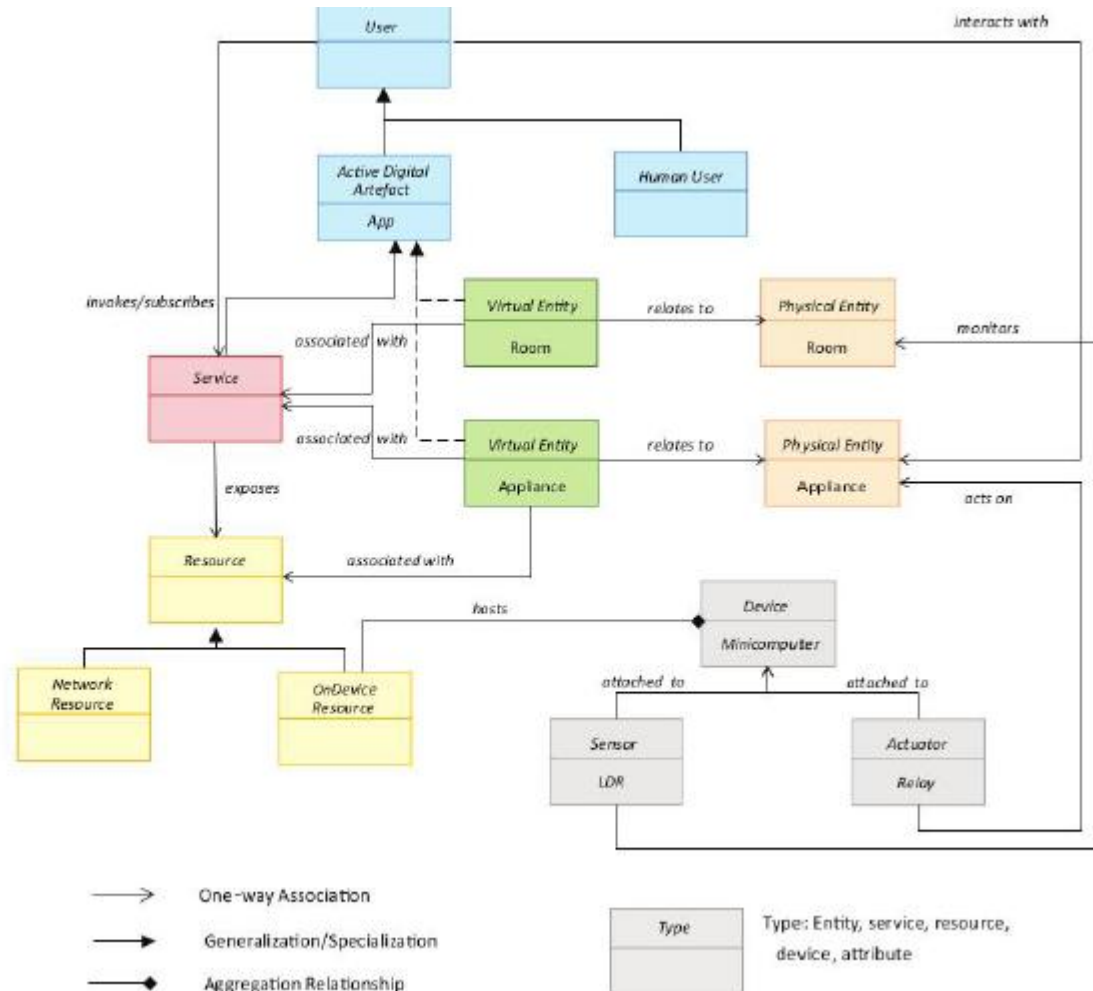
Domain Model Specification

- It represents the main concepts, entities and objects in the IoT system.
- It is independent of technology and platform.
- Defines the attributes of objects, relationship between objects
- It is an abstract representation of the concepts, objects and entities.
 - Physical Entity
 - Virtual Entity
 - Device
 - Resource
 - Service

Domain Model Specification

- **Physical Entity**
 - Discrete and identifiable entity of physical environment
 - Home automation system physical entities like room and light appliance.
- **Virtual Entity**
 - Representing the digital form of physical entity.
 - For each physical entity there was one virtual entity in the model.
 - Virtual entity for room is monitoring and for appliances is controller.
- **Device**
 - Interaction between physical entity and virtual entity.
 - Devices for gathering the information about physical entities
 - For home automation system mini computer is attached with light sensor and actuator.
- **Resource**
 - Software component information for on-devices and network resources.
 - The software for on devices will provide the information of the physical entity once it is actuated upon.
 - Similarly the software enables the network resources like databases.
 - For home automation system, the operating system is a software component that runs on the mini computer.
- **Service**
 - An interface for interacting with physical entity.
 - The service access the resource hosted on the device or the network resource to obtain information about the physical entity or perform actuation upon physical entity.

Domain Model for Home automation IoT System



Information Model Specification

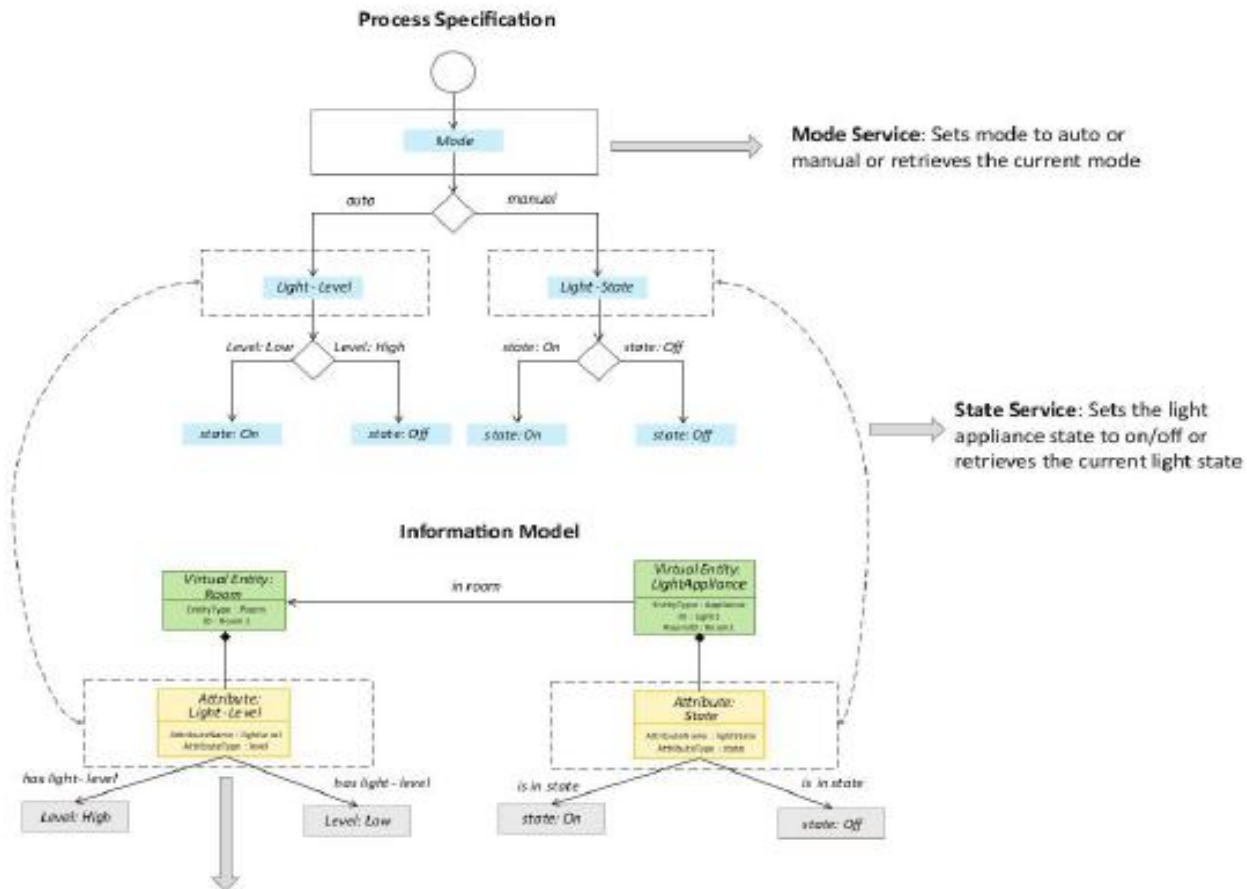
- The structure of all information in the IoT System like attributes, virtual entities, relations etc.,
- It adds more details to the virtual entities through their attributes and operations.

Information Model for Home automation IoT System

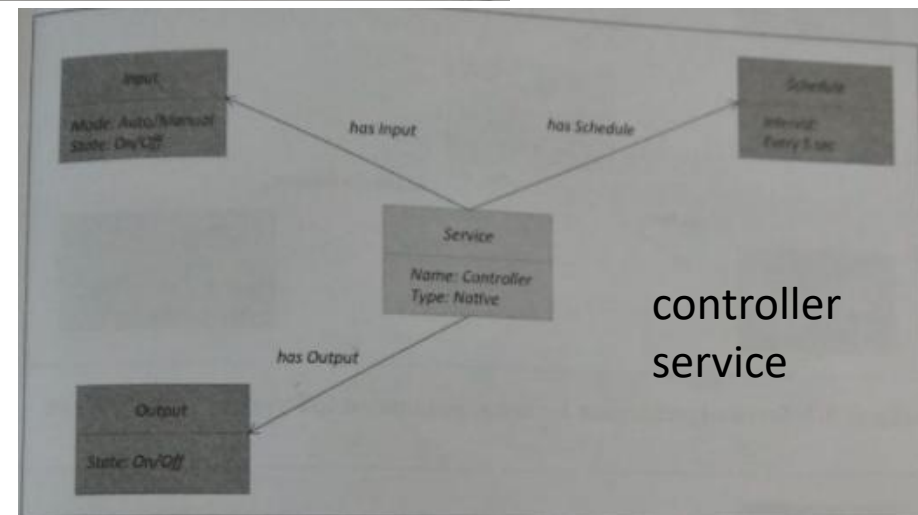
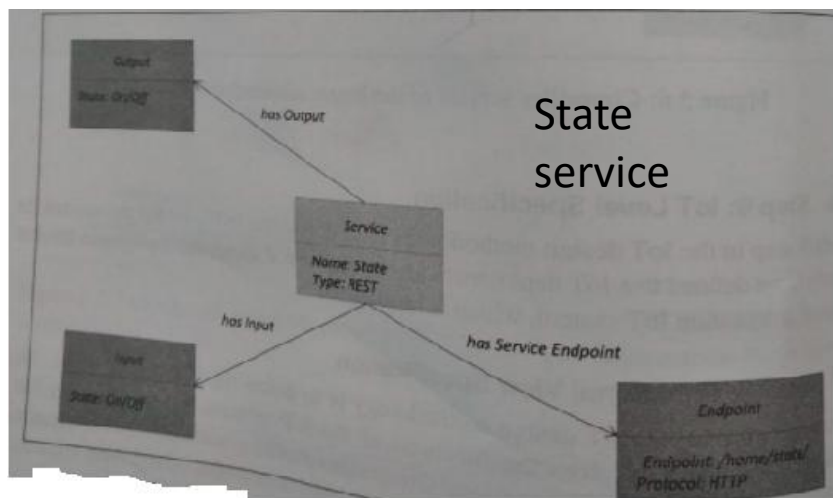
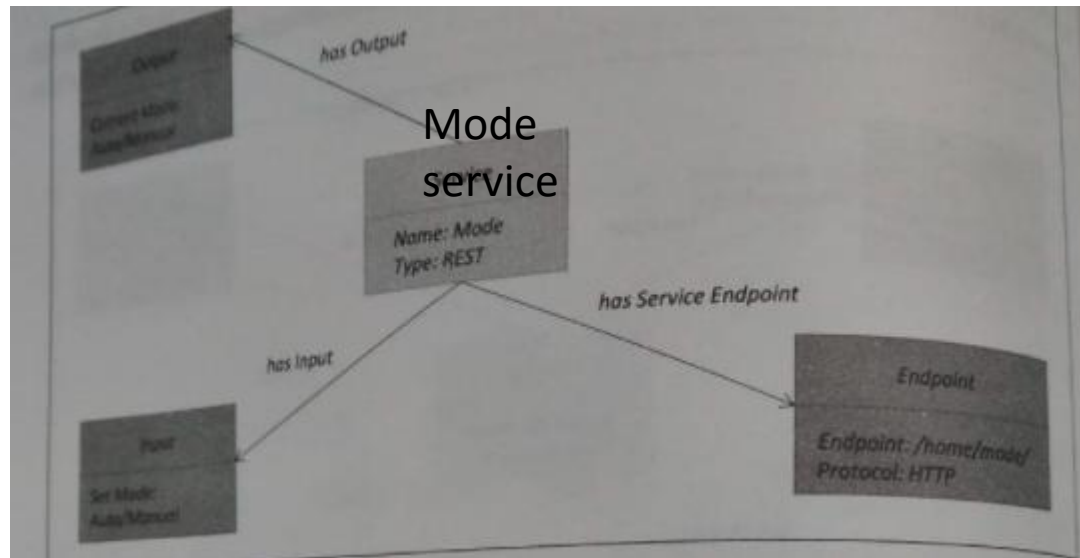


Service Specification

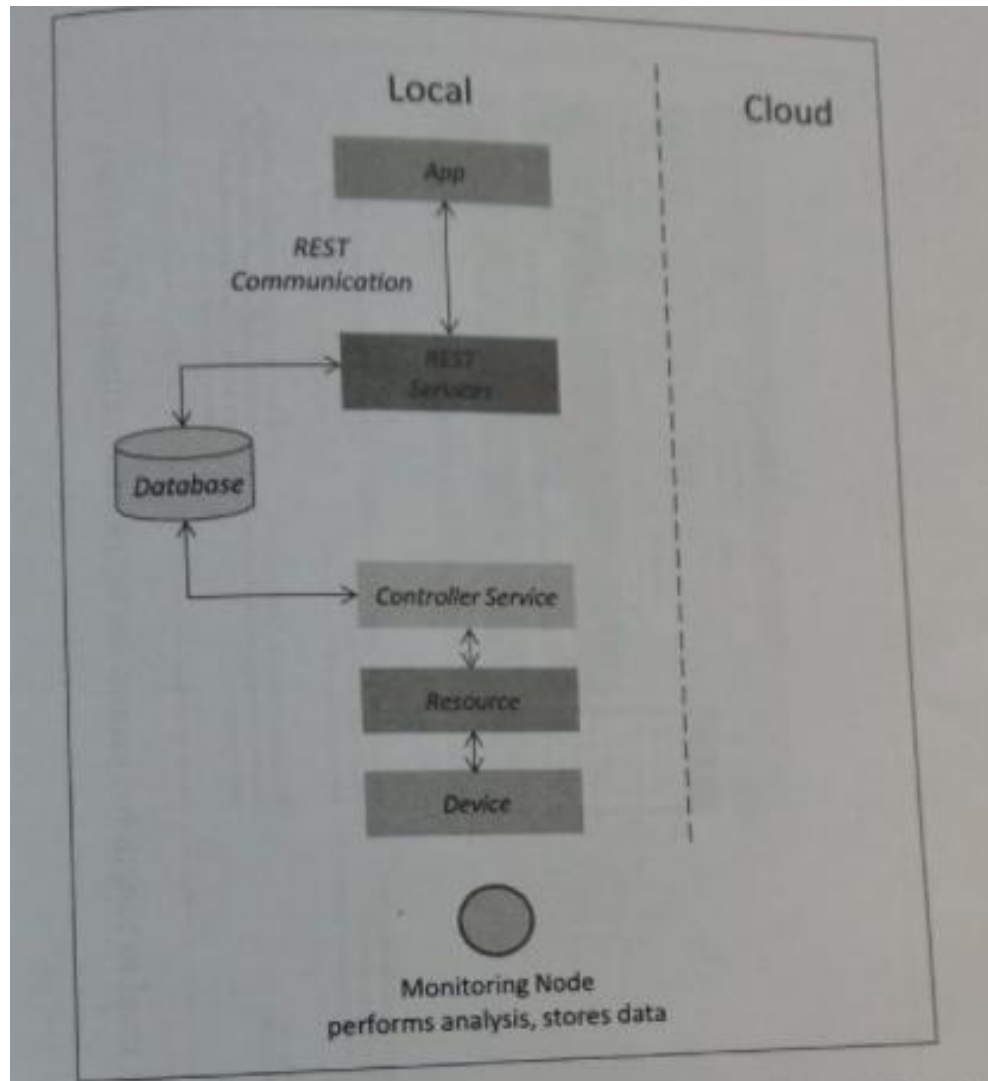
- Specifications about services like service schedules, service preconditions and service effects.
- From process specifications and information model states and attributes are identified.
- For Each state and attribute a service will be defined.



Service Specification for Home automation IoT System



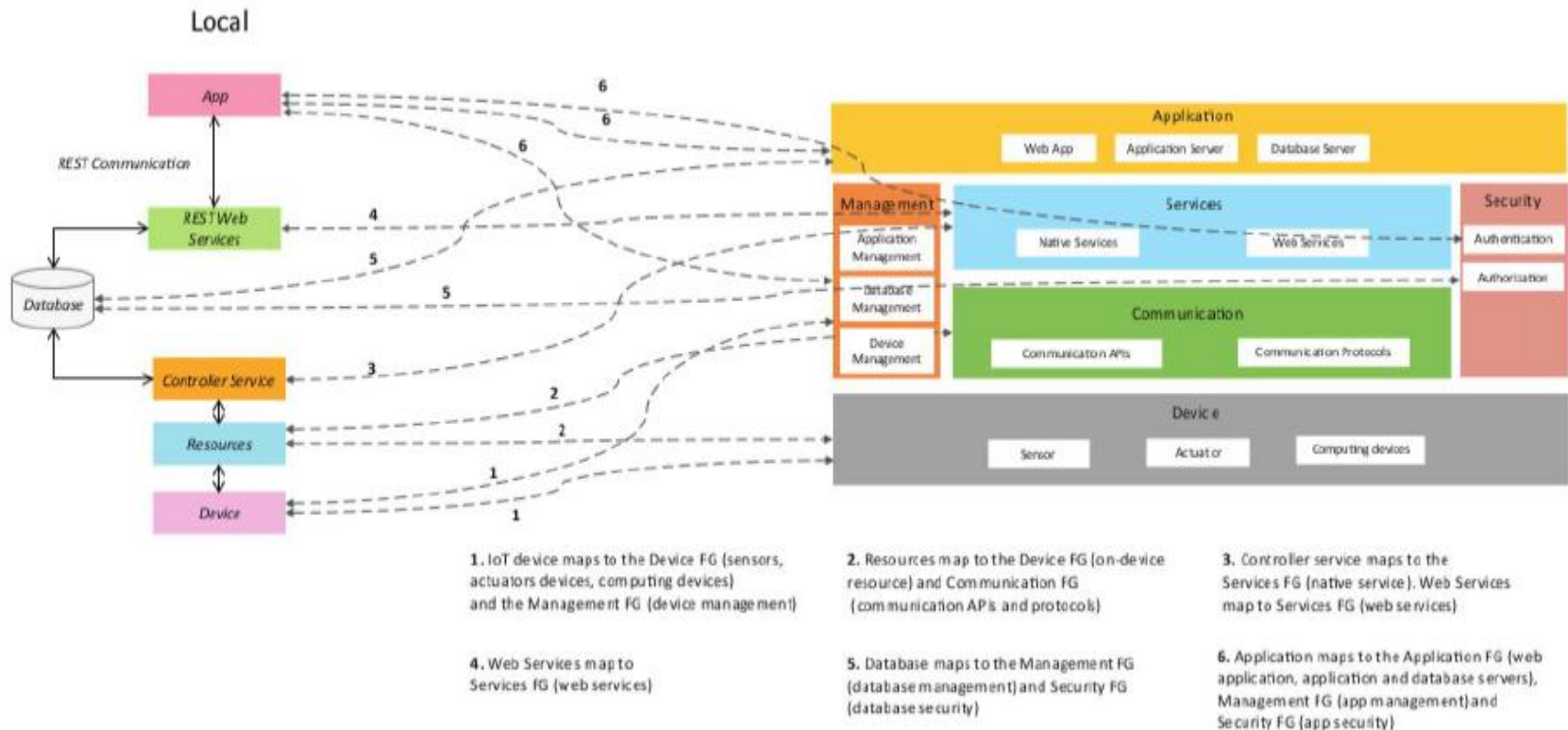
IoT Level specification



Function level specification

- About the functions of IoT system grouped into various functional groups(FG).
- Each FG provides functionalities to interact with the instances of concepts defined in Domain model or provides information related to the concepts.
- FG are
 - Device
 - Contains devices for monitoring and control.
 - For home automation system single board mini-computer, light and a relay switch.
 - Communication
 - Communication of IoT system like communication protocols that enables network connectivity.
 - Communication APIs
 - Services
 - Various services for device monitoring, device control service, data publishing and service discovery services.
 - REST and native services used for home automation system.
 - Management
 - Functionalities required for authentication and manage the IOT system.
 - Security
 - Security like authentication, data security and authorization.
 - Application

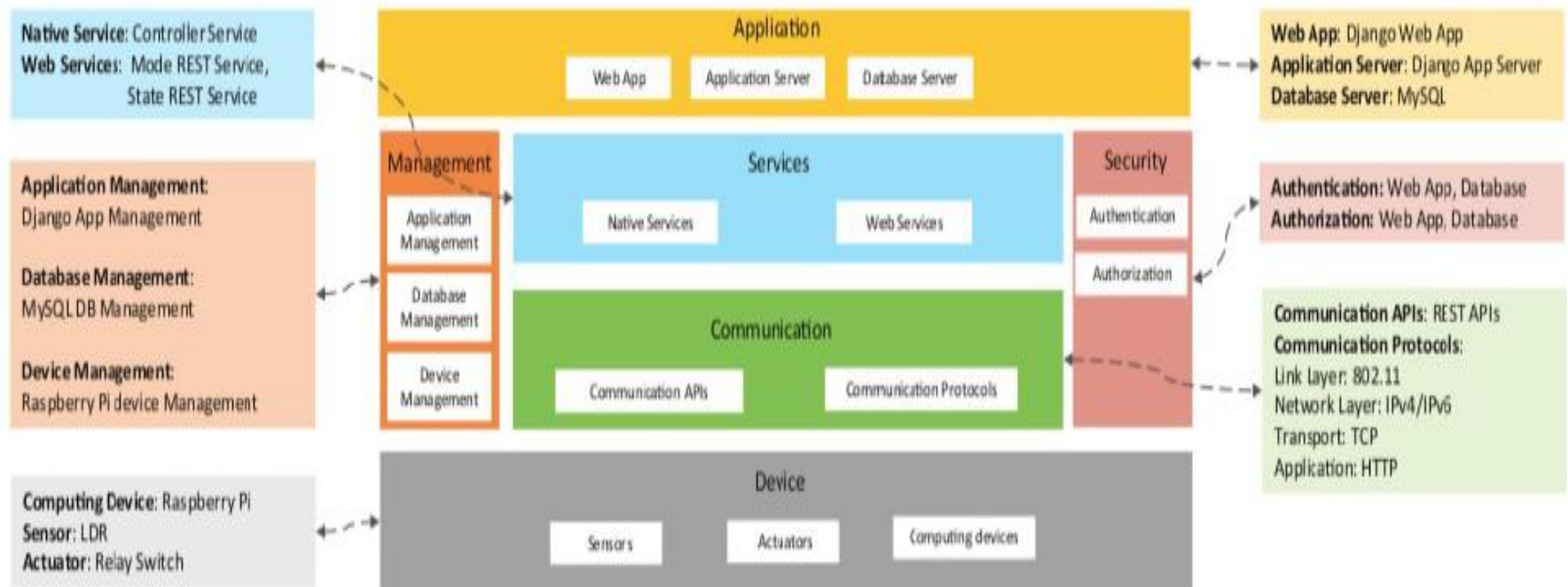
Function level specification for home automation system



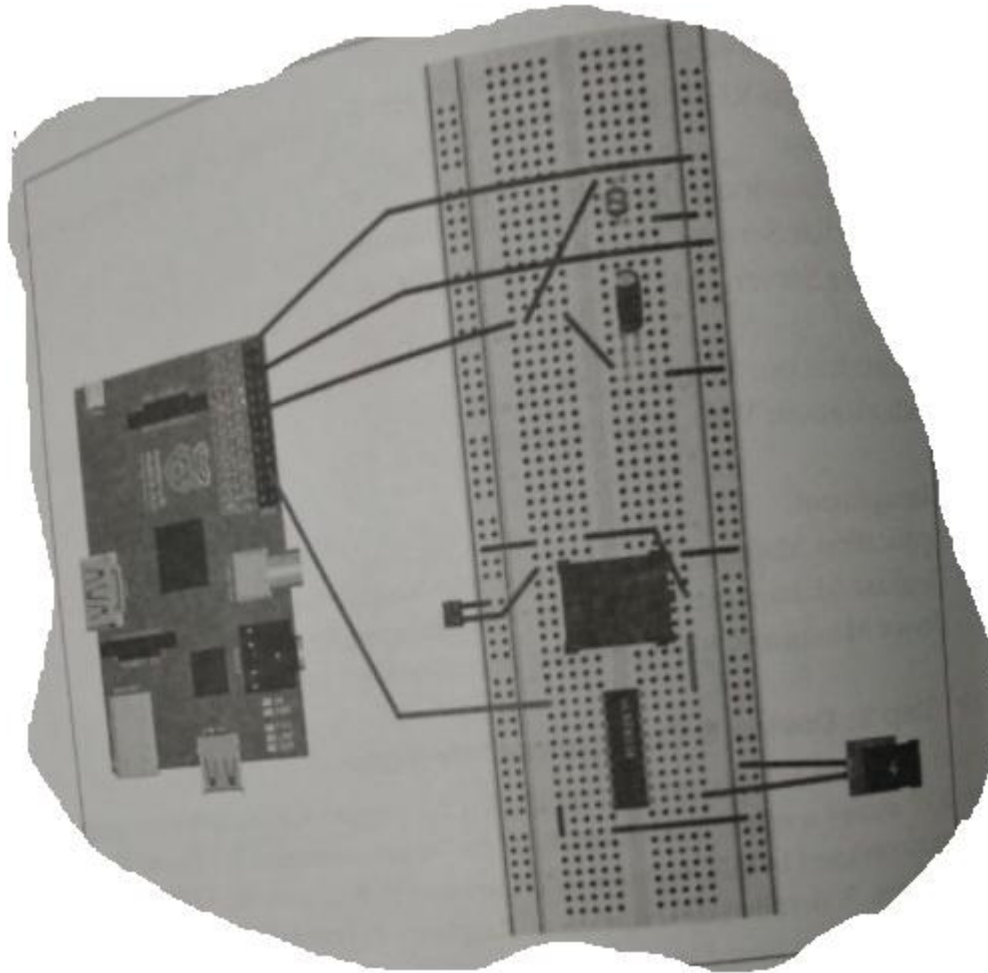
Operational view specification

- Options for pertaining to IoT system deployment and operation
 - Service hosting options, storage options, device options, application hosting options etc.,
- Devices: computing device (Raspberry Pi), light dependent sensor. Communication APIs, REST API & Communication Protocols: Link Layer-802.11, Network Layer-IPV4/IPv6,
- Services
 - Control services : Hosted on devices implemented in Python and run as a native service.
 - Mode services
 - State service:
- Applications
- Security
- Management

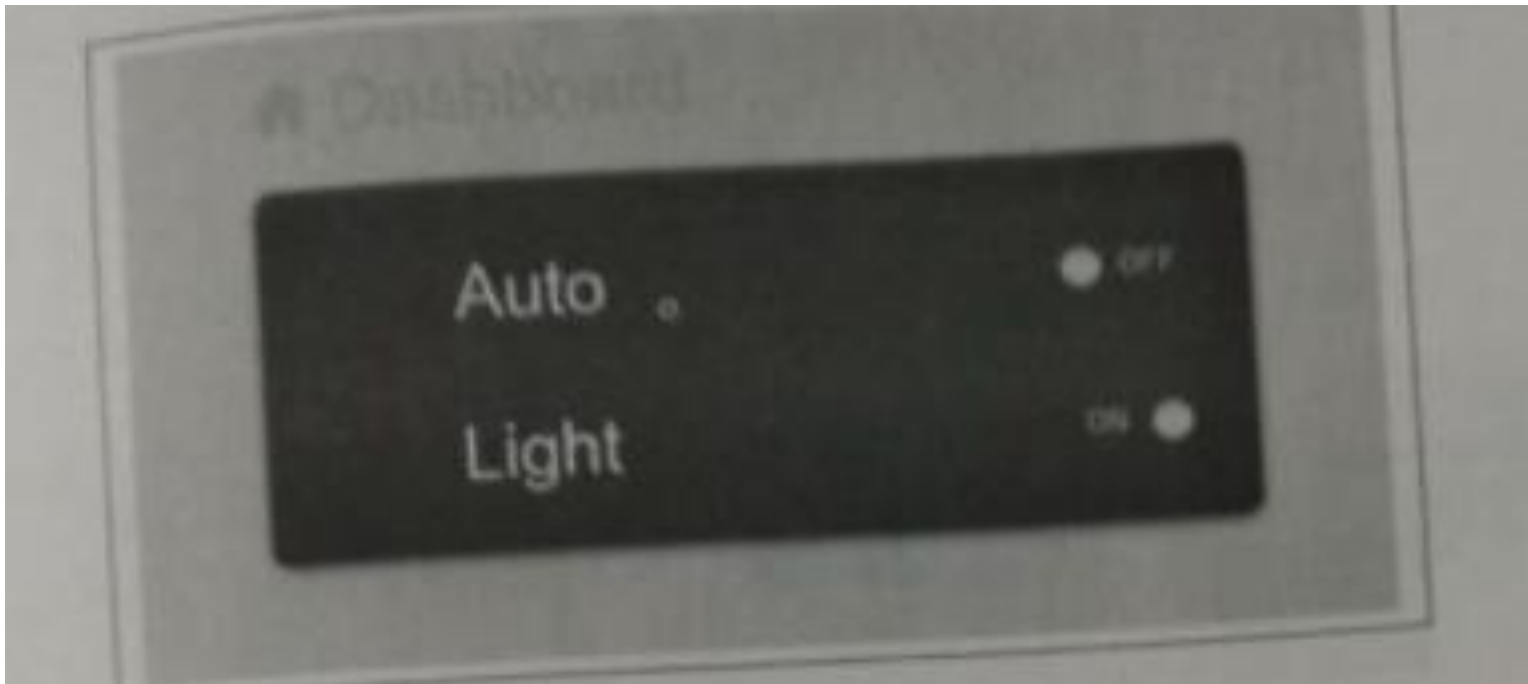
Operational view specification for home automation IoT system



Device & component integration



Application Development



DA-I

- Implementing a Smart Water Leakage Detection and Management System using IoT technology can significantly improve the city's water infrastructure efficiency. The System uses water leak identification & Controlling ,Indicating about location coordinate and other parameters to cloud for remote access. Design all levels of information modeling based on the given scenario

References

- <https://internetofthingsagenda.techtarget.com/blog/IoT-Agenda/Emerging-frameworks-for-cross-silo-IoT-data-models>
- https://www.w3schools.com/xml/xml_rdf.asp#:~:text=RDF%20stands%20for%20Resource%20Description,RDF%20is%20written%20in%20XML.
- [OWL Web Ontology Language Overview \(w3.org\)](http://www.w3.org/2002/07/owl/)
- https://en.wikipedia.org/wiki/Information_model
- https://en.wikipedia.org/wiki/Building_information_modeling
- [https://gilbane.com/artpdf/GR10.1.pdf/What is an Information Model Why do You Need One.html.](https://gilbane.com/artpdf/GR10.1.pdf/What%20is%20an%20Information%20Model%20Why%20do%20You%20Need%20One.html)
- <https://www.guru99.com/data-modelling-conceptual-logical.html>
- <https://afteracademy.com/blog/what-is-data-model-in-dbms-and-what-are-its-types>